```
<partial name="_ValidationScriptsPartial" />
}
```

Once this is done, launch the application, and navigate to /roles-manager/assign. Assign anna@test.com to the `Admin` role. Having done that, go back to the Assign .cshtml file and add a `using` directive for `Microsoft.AspNetCore.Authorization`. Then add an `Authorize` attribute to the `AssignModel` class, but this time, assign `"Admin"` as the value in the `Roles` property:

```
[Authorize(Roles = "Admin")]
public class AssignModel : PageModel
{
```

Rerun the application, and this time, sign in using either colin@test.com or paul@test.com. Then attempt to navigate to /roles-manager /assign. You should find you are redirected to the Access Denied page.

Log out, and then sign in with anna@test.com. This time, when you navigate to roles-manager /assign, you should access the page. Anna can access the page because she is a member of the role specified in the `Roles` property, whereas Paul is not. Although you have only passed one role name in, the `Roles` property takes a comma-separated list of role names, which provides an element of flexibility when using roles. In addition, users can belong to more than one role.

10.2.4 Using policies to apply role checks

You have extended the `Authorize` attribute to check that the current user belongs to the specified role. If you want to apply this check over the contents of an entire folder, you can use an overload of the `AuthorizeFolder` method that takes a *policy*; we will look at policies